

ECARMF Platform Kernel

Server Migration Guide

Moving the complete system — application, database, configuration — to another server

July 2026 · every step verified on the live system (restore drill proven: 28 tenants recovered)

Contents

Contents.....	2
Target server prerequisites.....	4
Method A — Copy binaries + restore database (recommended, ~30 min, no build tools).....	5
Step 1 — On the OLD server: take a fresh backup and gather the payload	5
Step 2 — On the NEW server: restore the database	5
Step 3 — Copy the application.....	5
Step 4 — Review the machine config	6
Step 5 — Install the service (elevated)	6
Step 6 — Register monitoring + open the firewall (elevated)	6
Step 7 — Verify (the acceptance checklist)	6
Step 8 — Cutover	7
Method B — Rebuild from source (when you want the target to build its own releases).....	8
Gotchas (each one learned the hard way).....	9

Everything the platform is lives in **three things**: the published app folder, the SQL database, and one machine-config file. Move those three, register the service, and the new server *is* the platform — every tenant, package, skill, record, score, audit entry, and access key carries over in the database.

What	Where on the current server	How it moves
Application (API + built admin UI)	C:\ECARMF\app (~75 MB, includes wwwroot)	Copy folder (or rebuild from source — Method B)
Database (all tenants & data)	SQL Express localhost\SQLEXPRESS, DB ECARMF_Kernel	Backup → copy .bak → restore
Machine config	C:\ECARMF\app\appsettings.Production.json	Copy + edit for the target
Operational scripts	repo deploy\ folder	Copy alongside
Windows service + tasks	service ECARMF, backup/health tasks	Re-registered on the target (scripts do it)
HTTPS certificate (if used)	your PFX + its password	Copy PFX securely; password via config/env
Platform AI key (if set)	env var ANTHROPIC_API_KEY	Re-set on the target (never in files)

What does NOT need copying: access keys (hashed in the DB — clients keep the same keys), tenant AI settings (in the DB), in-memory registries (rebuilt automatically at startup from active packages).

Target server prerequisites

1. **Windows 10 version 2004+ or Windows Server 2022+** — the app targets `net8.0-windows10.0.19041.0` (it uses Windows built-in OCR for document extraction).
2. **ASP.NET Core Runtime 8.0 (x64)** — the publish is framework-dependent; without this the exe exits immediately. Install the "ASP.NET Core Hosting Bundle" or "ASP.NET Core Runtime" from <https://dotnet.microsoft.com/download/dotnet/8.0>.
3. **SQL Server Express 2019+** installed as the named instance ``SQLEXPRESS`` (the default in the Express installer). If you must use a different instance/edition, you'll edit one line of config in step 4.
4. An **Administrator** PowerShell for the registration steps.

Method A — Copy binaries + restore database (recommended, ~30 min, no build tools)

Step 1 — On the OLD server: take a fresh backup and gather the payload

```
# Fresh, restore-verified backup (script already proven on this system)
powershell -File "deploy\backup-nightly.ps1"
powershell -File "deploy\backup-nightly.ps1" -VerifyRestore # proves the .bak is good

# Gather into one folder to carry over (USB / network share / scp):
# C:\ECARMF\backups\ECARMF_Kernel-<newest>.bak
# C:\ECARMF\app\ (the whole folder)
# C:\ECARMF\app\appsettings.Production.json (it's inside app\, comes along)
# <repo>\deploy\ (all the .ps1 scripts + runbooks)
# your HTTPS PFX (if any)
```

Freeze note: from this moment until cutover, treat the old server as read-only for real work — anything entered on the old server after the backup will not exist on the new one.

Step 2 — On the NEW server: restore the database

Copy the .bak to C:\ECARMF\backups\, then in an elevated PowerShell:

```
New-Item -ItemType Directory -Force C:\ECARMF\backups, C:\ECARMF\logs, C:\ECARMF\app | Out-Null

sqlcmd -S localhost\SQLEXPRESS -I -Q "
RESTORE FILELISTONLY FROM DISK='C:\ECARMF\backups\ECARMF_Kernel-<stamp>.bak';"
# note the two LogicalName values (typically ECARMF_Kernel and ECARMF_Kernel_log), then:

sqlcmd -S localhost\SQLEXPRESS -I -Q "
RESTORE DATABASE [ECARMF_Kernel]
FROM DISK='C:\ECARMF\backups\ECARMF_Kernel-<stamp>.bak'
WITH MOVE 'ECARMF_Kernel' TO 'C:\Program Files\Microsoft SQL
Server\MSSQL16.SQLEXPRESS\MSSQL\DATA\ECARMF_Kernel.mdf',
MOVE 'ECARMF_Kernel_log' TO 'C:\Program Files\Microsoft SQL
Server\MSSQL16.SQLEXPRESS\MSSQL\DATA\ECARMF_Kernel.ldf',
REPLACE;"

sqlcmd -S localhost\SQLEXPRESS -I -Q "SELECT COUNT(*) AS tenants FROM
ECARMF_Kernel.dbo.Tenants;"
# expect your tenant count (28 at time of writing)
```

(Adjust the MSSQL16.SQLEXPRESS path to whatever your SQL version created — check under

C:\Program Files\Microsoft SQL Server\.)

Step 3 — Copy the application

Place the copied app folder at `C:\ECARMF\app` (same path — the service definition, scripts, and backup paths all assume it). Put the deploy scripts at C:\ECARMF\deploy\.

Step 4 — Review the machine config

Open `C:\ECARMF\app\appsettings.Production.json`:

- **Connection string**: correct as-is for a default `localhost\SQLEXPRESS`. If your instance name differs, change `Server=localhost\SQLEXPRESS` accordingly.
- **'Security.AllowHeaderIdentity'**: `false` for production (key sign-in only). Set `true` only for an open test period.
- **HTTPS** (optional): add the `Kestrel` section per `RUNBOOK-golive-and-ai.md` §C2, pointing at your copied PFX.

Step 5 — Install the service (elevated)

```
C:\ECARMF\deploy\install-windows-service.ps1
Get-Content C:\ECARMF\install-result.txt      # expect: OK - service ECARMF is Running...
```

This one script does the heavy lifting: grants NETWORK SERVICE db_owner on the restored database, sets folder ACLs for backups/logs, creates the ECARMF service (auto-start, restart-on-failure, `http://*:5099` so it serves the whole LAN), starts it, and registers a nightly backup.

It registers a backup task pointing at `C:\ECARMF\backup-nightly.ps1` — copy `deploy\backup-nightly.ps1` there, or run `C:\ECARMF\deploy\register-backup-task.ps1` afterwards to use the deploy-folder copy (either works; don't run both).

Step 6 — Register monitoring + open the firewall (elevated)

```
C:\ECARMF\deploy\register-health-probe.ps1
New-NetFirewallRule -DisplayName "ECARMF Platform" -Direction Inbound -Protocol TCP -
LocalPort 5099 -Action Allow
# add 5443 too if HTTPS is configured
```

If a platform-wide Anthropic key was in use:

```
[Environment]::SetEnvironmentVariable('ANTHROPIC_API_KEY','sk-ant-...','Machine');
Restart-Service ECARMF
```

Step 7 — Verify (the acceptance checklist)

```
Invoke-WebRequest http://localhost:5099/health      -UseBasicParsing # 200 healthy
Invoke-WebRequest http://localhost:5099/health/ready -UseBasicParsing # 200 ready (DB
reachable)
Invoke-WebRequest http://localhost:5099/auth-mode   -UseBasicParsing # confirms
security mode
```

Then in a browser at `http://new-server:5099`:

- Sign in with an **existing operator access key** (keys migrated inside the DB).
- Spot-check: Clients list shows all tenants → open one tenant → Dashboard/Risk Register populated → Platform Risk totals match the old server → Audit Trail shows history.
- Prove backups on the new machine: `C:\ECARMF\deploy\backup-nightly.ps1 -VerifyRestore`.

Step 8 — Cutover

- Point users at `http://new-server:5099` (update the desktop shortcut `.url` files, bookmarks, and any integration feed URLs that push into the platform).
- **Stop the old server's service/process** (`Stop-Service ECARMF`) so no one keeps writing to the old database. Keep the old machine intact for a week as the rollback: if anything is wrong, stopping the new service and restarting the old one restores the exact pre-migration state.

Method B — Rebuild from source (when you want the target to build its own releases)

On the target, additionally install **.NET 8 SDK** and **Node.js 20+**, then:

```
git clone https://github.com/magdynamics/ECARMF.git ; cd ECARMF
cd frontend\admin-ui ; npm ci ; npm run build ; cd ..\..
dotnet publish src\ECARMF.Kernel.Api\ECARMF.Kernel.Api.csproj -c Release -o C:\ECARMF\app
dotnet test          # optional: 331 tests should pass
```

Then continue from **Method A step 2** (restore DB) and steps 4–8. Future updates on the target:

build → `deploy\update-app.ps1` (it already protects `appsettings.Production.json`).

Gotchas (each one learned the hard way)

5. **Exe exits instantly on the new server** → ASP.NET Core Runtime 8 is missing (framework-dependent publish). Install the runtime, not just .NET Desktop.
6. **"Connection string 'ECARMF' is not configured"** → the service/exe wasn't started from `C:\ECARMF\app` as its working directory, or `appsettings.Production.json` wasn't copied. The service installer sets this correctly; manual runs must `cd C:\ECARMF\app` first.
7. **Login failed for NT AUTHORITY\NETWORK SERVICE** → step 5's installer grants it; if you restored the DB *after* installing the service, re-run the installer (idempotent).
8. **`http://hostname:5099` fails but `localhost` works** → binding must be `http://*:5099` (the installer does this); also check the firewall rule from step 6.
9. **Old Windows** → WinRT OCR requires Windows 10 2004+/Server 2022; on older servers the app won't run (the Infrastructure project targets `windows10.0.19041`).
10. **Deploys clobbering config** — never mirror without `/XF appsettings.Production.json` (`update-app.ps1` already includes it).
11. **Don't run two live servers.** Keys work on both, users won't notice they're writing to different databases, and the histories diverge irreversibly. Back up → freeze → restore → verify → cut over → stop the old one.
12. **Time:** all timestamps are UTC `datetimeoffset` — a different server time zone changes nothing.